

# Direct Human Read/Write Access: The Second Tool-Agnosticism Requirement as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

**Author:** Wenxin Li (Independent Researcher)

**ORCID:** 0009-0004-8065-3235

**Date:** 2 May 2026

**License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the three minimal tool-agnosticism requirements named in the source paper — Requirement 2, **direct human read/write access** — as a standalone architectural commitment with independent operational content, separable from Requirements 1 and 3 with which it composes into the integrated tool-agnosticism commitment.

## Abstract

The Coordination Knowledge Substrate (CKS) pattern's "tool-agnostic" commitment names three minimal requirements that any environment must satisfy to host a CKS substrate: persistent structured state, direct human read/write access, and LLM access to substrate content. A separate note formalizes the joint commitment. This note formalizes the second of the three as having independent operational content that can be defended, implemented, and tested independently of Requirements 1 and 3. The motivation is concrete: implementations under pressure to add governance features, AI-powered interfaces, or workflow-driven access drift toward host environments that route human access through additional layers, each of which feels like an improvement and each of which erodes the architectural commitment to direct access. Naming Requirement 2 as standalone gives downstream implementers a precise specification of what host environments must provide for human governance to be architecturally — not merely procedurally — exercisable. The note states the requirement's five operational components, identifies what it does and does not require, distinguishes it from four adjacent host-capability patterns, names the connections that make it load-bearing for several other CKS commitments, enumerates the failure modes that violate it specifically, and provides an operational test for Requirement 2 separable from the other two minimal requirements.

## 1. Why Requirement 2 needs to be formalized as standalone

The CKS pattern's tool-agnosticism commitment names three minimal requirements together (§7.1 of the source paper), and the parent foundational note treats them as the joint host-interface specification. The joint framing is correct as far as it goes, and this note does not contradict it. But it leaves a class of concerns architecturally underspecified, particularly around host environments that satisfy the joint commitment in some respects while failing Requirement 2 specifically.

The motivating cases are concrete. Environments that provide read access but gate writes through workflow systems satisfy the read direction while failing the write direction. Environments that provide direct access only through admin interfaces fail directness when the admin tooling itself becomes a precondition. Environments that route all human access through LLMs or governance dashboards fail directness in both directions. Environments that grant access permission but require specialized runtime mediation make exercise of authority dependent on the runtime layer's continued cooperation. Without standalone treatment, the architecture has no principled vocabulary for distinguishing which sub-component is failing.

A second motivation is the strategic prior-art posture. Patentable derivations of CKS that focus on substrate access architectures — admin interface designs, governance UI patterns, LLM-mediated access systems, workflow-gated substrate operations — are more defensibly contested when Requirement 2 is publicly formalized as standalone, because any "substrate access innovation" can be evaluated against the specific commitment to direct human read/write access in inspectable form rather than against an undifferentiated bundle.

A third motivation, and the most consequential one architecturally, is the connection to the human-governed commitment from the source paper's §3.3. The three rights named there — inspect, modify, override — are exercisable in the architectural sense only because Requirement 2 grants direct access. Without Requirement 2, the three rights become procedural commitments rather than architectural ones: humans may have nominal authority but cannot operationally exercise it without going through whatever gating layer the host imposes. Naming Requirement 2 as standalone is what makes the human-governed commitment architecturally defensible against implementations that grant authority on paper but gate exercise through runtime middleware.

## 2. The Requirement 2 commitment, defined precisely

A host environment satisfies **Requirement 2** if and only if all of the following hold during the substrate's existence on the host. Authority scope — which humans hold which permissions over which substrate content — is determined by the deployment per the broader human-governed authority architecture; Requirement 2 specifies that wherever access is granted, the access is direct.

**(a) Direct read access.** Humans with appropriate access scope can read substrate content directly. The path from human to substrate content does not pass through an LLM call, an embedding query, an agent framework, or any other mediating layer as a precondition of access. The human reads substrate content in the form it exists in substrate state.

**(b) Direct write access.** Humans with appropriate access scope can write substrate content directly. The path from human to substrate write does not pass through an LLM, an approval workflow, or a specialized runtime as a precondition. The human's write takes effect as substrate state, with the human as the attributed writer.

**(c) Inspectable form.** Substrate content reaches humans in inspectable form — a form that preserves substrate state's authoritative meaning. Embedded representations, LLM-generated summaries, derived views, or other transformations may be available alongside direct access; what the architecture commits to is that direct access to underlying substrate state in inspectable form is available when humans choose to exercise it.

**(d) Commodity tools.** The access path does not require specialized governance tooling. Humans can exercise read and write access through commodity tools that already meet Requirement 1 — the spreadsheet that hosts the substrate, the database client that connects to substrate storage, the document editor that opens substrate files. Vendor-specific governance dashboards, AI-powered access interfaces, or other specialized runtimes are not preconditions of access.

**(e) No architectural gating.** The access path is not architecturally gated by approval, scheduling, or workflow systems. Humans exercising access within their authority scope do so on their own initiative; the architecture does not impose process gates as preconditions. Process layers may exist as deployment additions — review queues, change-management forms, audit trails — but they cannot be architectural preconditions. Components (a) and (b) name the read- and write-direction specializations of this commitment; (d) names its tooling specialization; (e) is the umbrella architectural-property commitment that grounds all three.

The five components together define what Requirement 2 requires architecturally. A host that satisfies fewer than five components fails the requirement, regardless of what other useful capabilities it provides.

### 3. What Requirement 2 does NOT require

The standalone treatment is not a maximalist treatment. Stating precisely what Requirement 2 does not require keeps the standalone framing from drifting into something stronger than the source paper supports.

**It does not require every human to have access to every piece of substrate content.** Access scope is a deployment configuration; authority partitioning, role-based access controls, and per-scope restrictions are deployment concerns the architecture supports without specifying. What Requirement 2 requires is that for any human granted access, the access is direct rather than mediated.

**It does not require any specific user interface.** Spreadsheet UIs, database clients, document editors, command-line access, programmatic access, or any other interface form is admissible, provided access is direct and substrate state is in inspectable form.

**It does not forbid LLM-assisted tooling alongside direct access.** Humans may use LLM-powered search, summary, or navigation tools, and such tooling is often valuable. What Requirement 2 forbids is the LLM being a *precondition* of access. LLM tools as optional adjuncts are fully Requirement 2-compliant.

**It does not forbid audit logging.** Logs that record access events do not violate the requirement; what would violate it is logging that gates access — i.e., access denied unless the logging system itself functions correctly.

**It does not forbid display transformations.** Interfaces may render substrate content with formatting, syntax highlighting, layout adjustments, or visual styling. Display rendering on top of inspectable state is fine; the commitment is to direct access to substrate state in inspectable form, not to a particular display.

**It does not forbid permission systems.** Authentication, authorization, and access control are normal host capabilities. What Requirement 2 forbids is specialized governance gating beyond standard permissions — once a human is authenticated and authorized, access is direct, not further mediated.

## 4. What Requirement 2 is NOT

Four adjacent host-capability patterns are commonly conflated with Requirement 2. Each is a real and reasonable host capability in some other architecture; naming what Requirement 2 is not is what prevents the misreading.

**Not admin interfaces with read access.** Admin interfaces satisfy Requirement 2 only if the access they grant is direct (not mediated by application logic that transforms or filters the data) and includes write access (not just read). An admin interface that displays content in transformed form, or that allows reads but routes writes through application validation layers as preconditions, fails the requirement even though it provides "human access" in some sense.

**Not query APIs over opaque storage.** Query APIs (REST, GraphQL, SQL) satisfy Requirement 2 only if queries return substrate content in inspectable form (not opaque representations) and the API supports both read and write directly (not routed through application logic as preconditions). An API that returns embedded representations, exposes only some fields, or routes writes through validation layers as architectural preconditions fails the requirement.

**Not governance dashboards built on top of substrates.** Dashboards that present substrate content for human review, approval workflows, and decision-making may be useful, but they fail Requirement 2 if they are the only path to human access. The architectural commitment is to direct access in inspectable form, not access through dashboards that transform, filter, or curate substrate content for the human.

**Not LLM-mediated access systems.** Some "AI-powered" interfaces position the LLM as the human's interface to substrate content — the human asks the LLM what the substrate contains, requests LLM-generated summaries, or instructs the LLM to make substrate writes. Such access fails Requirement 2 because the LLM is a precondition. The architecture forbids this gating regardless of how sophisticated or human-friendly the LLM interface is. LLM-assisted access alongside direct access is permissible; LLM-only access is not.

## 5. Why Requirement 2 is load-bearing for downstream commitments

Requirement 2 is unusually load-bearing within the CKS commitment set. Several other architectural commitments depend on it specifically; naming each connection makes the dependence visible to downstream implementers.

**The human-governed commitment.** The three rights — inspect, modify, override — that the source paper commits to at §2.1 and §3.3 are operationally exercisable only because Requirement 2 grants direct access. Without Requirement 2 the rights become procedural commitments rather than architectural ones: humans may have authority on paper but cannot exercise it without going through whatever gating layer the host imposes.

**The inspect right.** The right's direct-access component grounds in Requirement 2's read direction. The right requires that humans can read substrate content in inspectable form, without LLM intermediation as a precondition; this is precisely what the requirement commits the host to provide. The standalone formalization of the inspect right is in this sense architecturally downstream of Requirement 2: the right is the authority commitment, Requirement 2 is the host commitment that makes the authority exercisable.

**The modify right.** The modify right's direct-write component grounds in Requirement 2's write direction in the same way. Without it, the modify right reduces to a request submitted to a gating layer, which is not the architectural property the source paper commits to.

**The override right.** The override right's no-justification-as-precondition component depends on Requirement 2's no-architectural-gating component. Override authority is exercisable architecturally only because Requirement 2 commits the host to non-gated access; an override gated by an approval workflow is procedural override under another name.

**The non-specialist governance commitment.** The architectural property of governance accessibility in commodity tools (§7.4) depends on Requirement 2 specifically. Without component (d), governance becomes specialist work, accessible only to those trained in the specialized governance tooling the host imposes. A non-specialist using a spreadsheet to inspect or modify a substrate is exercising governance at full architectural scope precisely because the host has committed to making that access direct, ungated, and in inspectable form.

**The architectural-property qualifier on governance.** The distinction between architectural and procedural governance — that governance is a property of the system's design rather than of a particular workflow or vendor's policy — depends on Requirement 2 making human access architectural. Components (a), (b), and (e) together commit the host to direct, ungated access in inspectable form; without this, governance becomes whatever the gating layer permits today, which is procedural by definition.

These connections are not new commitments; they follow from treating Requirement 2 as having independent operational content that other CKS commitments depend on at specific, namable architectural points.

## 6. Failure modes that violate Requirement 2

A host environment can fail Requirement 2 specifically, even when it satisfies Requirements 1 and 3. Ten failure modes name the most common ways this happens in the 2024–2026 AI infrastructure landscape.

**(a) LLM-mediated read access.** The host requires humans to query substrate content through an LLM interface; direct access to substrate state is not available. Reading substrate content requires asking the LLM what the substrate contains.

**(b) LLM-mediated write access.** The host requires humans to submit write requests through an LLM that interprets them and produces substrate writes; direct write access is not available. The LLM is a precondition of writing.

**(c) Workflow-gated access.** Every human read or write must flow through approval workflows, scheduling systems, or change-management gates as architectural preconditions. Direct access is not available; even read access requires going through the gating layer.

**(d) Specialized-runtime gating.** All human access is routed through a specialized governance runtime, observability layer, or AI gateway. The gateway may enforce policies, log activity, or provide additional features; what makes it a violation is that it is a precondition of access rather than an optional layer.

**(e) Embedded-only representations.** Substrate content is stored in embedded form (vector representations, opaque binary encodings) and only embedded access is provided to humans. Humans cannot read substrate

state in inspectable form; they can inspect only numerical or opaque artifacts of substrate state.

**(f) Summary-only access.** The host provides only LLM-generated summaries or curated views; direct access to underlying state is not available. Humans see only what the summary system chose to surface.

**(g) Governance-dashboard-only access.** The host provides only a governance dashboard that displays substrate content in a curated, transformed, or workflow-oriented form. Direct substrate access is not available; humans must use the dashboard.

**(h) Read-only access without write.** The host provides direct read access but routes writes through application logic, validation layers, or approval workflows as architectural preconditions. Read direct, write mediated — half of Requirement 2 satisfied, half failing.

**(i) Permission-conditional access.** The host's permission system grants access "in principle" but the actual access path requires going through admin tools, support tickets, or other procedural mechanisms. The permission is nominal; the access is procedural rather than direct.

**(j) Vendor-revocable access.** The host's vendor or runtime middleware can in principle prevent direct human access by changing access controls, modifying interfaces, or updating policies. Even if direct access works in practice, the architectural commitment fails because the vendor holds revocation power; Requirement 2 is a property of the system's design, not of a vendor's current policy.

A host that exhibits any of (a)–(j) does not satisfy Requirement 2, even when it provides other useful capabilities and satisfies Requirements 1 and 3.

## 7. Operational test

A host environment satisfies Requirement 2 if and only if all of the following are true at all times during the substrate's existence on the host:

1. Humans with appropriate access scope can read substrate content directly, without LLM intermediation, runtime gating, or workflow gating as architectural preconditions.
2. Humans with appropriate access scope can write substrate content directly, with the same lack of architectural preconditions, and writes take effect as substrate state with the human as the attributed writer.
3. Substrate content reaches humans in inspectable form — direct access to underlying substrate state is available, not only embedded representations or LLM-curated views.
4. Access can be exercised through commodity tools that meet Requirement 1; specialized governance tooling is not required as a precondition of access.
5. The vendor, runtime middleware, or any non-architectural party cannot in principle revoke direct human access for humans with appropriate scope; the access is architectural rather than procedural.

A host that fails any of (1)–(5) does not satisfy Requirement 2, even if it provides human access in some other sense. The integrated three-requirement test from the parent foundational note applies separately and is formalized as joint sufficiency in a sibling note; the present test is for Requirement 2 alone.

## 8. Why naming Requirement 2 as standalone matters

Implementations under pressure to add governance features, AI-powered interfaces, or workflow-driven access consistently drift toward host environments that route human access through additional layers. The drift is steady because each layer feels like an improvement — better validation, richer interfaces, smarter assistance, clearer audit posture — and the substrate seems to "still work" with the layer in place. The architectural commitment to direct access is what each layer erodes when added as a precondition rather than as an adjunct.

Implementations that drift away from Requirement 2 produce systems where humans nominally have authority over substrate content but cannot operationally exercise it without going through whatever gating layer the host imposes. The downstream consequences manifest predictably: governance failures (the three rights become procedural rather than architectural), accessibility failures (commodity-tool governance becomes impossible because the gating layer requires specialist tools), dependency failures (governance becomes vendor-locked or middleware-locked because the gating layer is held by a particular vendor), and architectural-property failures (governance is resilient only when the gating layer functions correctly, which is the definition of a procedural rather than architectural property).

Naming Requirement 2 as a standalone architectural commitment — with the five components specified in §2, the limitations clarified in §3, the four adjacent-pattern distinctions in §4, the load-bearing connections in §5, the ten failure modes in §6, and the five-clause operational test in §7 — gives downstream implementers a precise specification of what host environments must provide for human governance to be architecturally exercisable. It distinguishes what the architecture commits to (direct access in inspectable form, ungated, in commodity tools) from what is left to deployment configuration (which humans hold which scopes, which interfaces are used, which optional layers sit alongside). And it names a specific axis on which downstream implementations can be evaluated independently of Requirements 1 and 3, allowing the diagnosis "Requirement 2 is failing" to land precisely rather than being absorbed into a general "tool-agnosticism is failing somehow."

A sibling note formalizes Requirement 1 (persistent structured state) as standalone; a subsequent note formalizes Requirement 3 (LLM access to substrate content); two further notes formalize the joint-sufficiency and individual-necessity properties of the three-requirement set as a whole. The five-note decomposition together gives the full operational content of the tool-agnosticism commitment, with each requirement defensible as a standalone architectural property and the integrated commitment defensible as the joint composition of the three.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "Requirement 2" — direct human read/write access in inspectable form, ungated, in commodity tools — in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

---

### **Source paper**

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

### **How to cite this note**

Li, W. (2026). *Direct Human Read/Write Access: The Second Tool-Agnosticism Requirement as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. 2 May 2026. ORCID: 0009-0004-8065-3235.